

Polymorphismus (Polymorphism)

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Definition	Polymorphismus erlaubt es, dass eine Methode oder eine Klasse unterschiedliche Formen annehmen kann.	Ermöglicht die Verwendung eines gemeinsamen Interfaces für verschiedene Implementierungen. Unterstützt Flexibilität und Erweiterbarkeit im Code.
Arten des Polymorphismus	1. Kompilierungszeit-Polymorphismus (Methodenüberladung). 2. Laufzeit-Polymorphismus (Methodenüberschreibung)	Methodenüberladung (<code>Method overloading</code>). Methodenüberschreibung (<code>Method overriding</code>).
Methodenüberladung (<code>overloading</code>)	Mehrere Methoden mit demselben Namen, aber unterschiedlichen Parametern in einer Klasse.	<code>void print(int x)</code> und <code>void print(double x)</code> .
Methodenüberschreibung (<code>overriding</code>)	Eine Methode in der Unterklasse überschreibt eine Methode der Superklasse, um ihr Verhalten zu ändern.	Verwendung von <code>@Override</code> , um sicherzustellen, dass eine Methode korrekt überschrieben wird.
Schlüsselwort <code>@Override</code>	Wird genutzt, um anzuzeigen, dass eine Methode eine geerbte Methode überschreibt.	<code>@Override public void makeSound() {}</code> → Methode überschreibt die Implementierung der Superklasse.
Polymorphismus mit Vererbung	Eine Unterklasse kann als Superklasse behandelt werden, aber sie kann eigene Implementierungen für bestimmte Methoden bereitstellen.	<code>Animal a = new Dog();</code> → <code>Dog</code> -Objekt wird als <code>Animal</code> behandelt. Aufruf <code>a.makeSound();</code> führt die <code>Dog</code> -Version der Methode aus.
Polymorphismus mit Interfaces	Eine Klasse kann mehrere Interfaces implementieren und unterschiedliche Implementierungen für deren Methoden bereitstellen.	<code>class Car implements Vehicle, Machine {}</code> → <code>Car</code> kann sowohl als <code>Vehicle</code> als auch als <code>Machine</code> behandelt werden.
super -Schlüsselwort in Polymorphismus	Wird verwendet, um auf die Methoden der Superklasse zuzugreifen, falls diese überschrieben wurden.	<code>super.someMethod();</code> ruft die Methode der Superklasse auf, auch wenn sie überschrieben wurde.
Abstrakte Klassen und Polymorphismus	Ermöglichen das Erzwingen einer gemeinsamen Schnittstelle für Unterklassen.	<code>abstract class Animal { abstract void makeSound(); }</code> → Jede Unterklasse muss <code>makeSound()</code> implementieren.
Interfaces und Polymorphismus	Erlaubt mehreren Klassen, eine gemeinsame Schnittstelle zu implementieren, ohne dass sie von einer gemeinsamen Klasse erben müssen.	-
instanceof -Operator	Prüft zur Laufzeit, ob ein Objekt eine Instanz einer bestimmten Klasse oder eines Interfaces ist.	<code>if (a instanceof Dog) { ((Dog) a).bark(); }</code> prüft, ob <code>a</code> ein <code>Dog</code> ist und ruft dann <code>bark()</code> auf.

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Casting (Upcasting & Downcasting)	Upcasting: Eine Unterklasse wird als Superklasse behandelt. Downcasting: Eine Superklasse wird explizit in eine Unterklasse umgewandelt.	<code>Animal a = new Dog();</code> (Upcasting, implizit). <code>Dog d = (Dog) a;</code> (Downcasting, explizit).
Polymorphismus mit Generics	Generische Klassen und Methoden können mit verschiedenen Datentypen verwendet werden.	<code>class Box<T> { T value; } → <code>Box<Integer> intBox = new Box<>();</code> kann für verschiedene Typen verwendet werden.</code>
default-Methoden in Interfaces	Interfaces können Standardimplementierungen bereitstellen, die in Unterklassen überschrieben werden können.	<code>interface A { default void show() { System.out.println("Default A"); } }</code> → Unterklasse kann <code>show()</code> überschreiben oder Standard behalten.
Polymorphismus in Collections (Java API)	Java-Klassenbibliotheken nutzen Polymorphismus intensiv, z. B. in der <code>List</code> -Schnittstelle.	<code>List<String> myList = new ArrayList<>();</code> → <code>ArrayList</code> wird als <code>List</code> behandelt.
Polymorphismus und Design Patterns	Viele Design Patterns basieren auf Polymorphismus, um flexible Architekturen zu ermöglichen.	Factory Pattern: Erzeugt Objekte basierend auf dem Polymorphismus einer gemeinsamen Schnittstelle. Strategy Pattern: Erlaubt das Wechseln von Implementierungen zur Laufzeit.
Performance-Überlegungen	Polymorphe Methodenaufrufe sind geringfügig langsamer als direkte Methodenaufrufe, da die JVM zur Laufzeit die korrekte Methode ermitteln muss.	JIT-Compiler optimiert polymorphe Methoden durch Inlining, um den Overhead zu reduzieren.

Polymorphismus in Java

ermöglicht es, dass dieselbe Schnittstelle unterschiedliche Implementierungen annehmen kann. Dies bedeutet, dass ein Objekt je nach Kontext unterschiedliche Formen annehmen kann.

- **Methodenüberladung (Overloading)**

Dies tritt auf, wenn mehrere Methoden denselben Namen, aber unterschiedliche Parameterlisten haben. Die korrekte Methode wird zur Kompilierzeit auf Basis der Parameter bestimmt. Java verwendet die Anzahl und den Typ der Parameter, um die richtige Methode zu wählen.

- **Methodenüberschreibung (Overriding)**

Hier überschreibt eine Subklasse eine Methode der Superklasse mit einer eigenen Implementierung. Bei Polymorphismus zur Laufzeit wird entschieden, welche Version einer Methode aufgerufen wird, basierend auf dem tatsächlichen Typ des Objekts, das die Methode aufruft, nicht auf dem statischen Typ der Referenz.

- **Dynamische Bindung**

Java verwendet dynamische Bindung (auch späte Bindung genannt) für Methodenaufrufe, die zur Laufzeit entschieden werden. Dies bedeutet, dass der tatsächliche Methodentyp erst zur Laufzeit bestimmt wird, was in einer Vererbungshierarchie mit überschriebenen Methoden entscheidend ist.

- **Polymorphismus durch Interfaces**

Polymorphismus tritt auch auf, wenn verschiedene Klassen dasselbe Interface implementieren. Das Interface definiert nur die Methodensignaturen, während die Implementierungen der einzelnen Klassen unterschiedlich sein können. Dies ermöglicht die Verwendung unterschiedlicher Implementierungen durch denselben Aufrufmechanismus.

- **`instanceof` Operator**

Der `instanceof`-Operator ermöglicht es, zur Laufzeit zu prüfen, ob ein Objekt eine Instanz einer bestimmten Klasse oder eines Interfaces ist. Dies wird häufig in Kombination mit Polymorphismus verwendet.

Zusammenfassung

Polymorphismus ist ein zentrales Konzept der objektorientierten Programmierung in Java, das Methodenüberladung, Methodenüberschreibung, Vererbung, Interfaces und Generics umfasst. Er ermöglicht flexible und erweiterbare Architekturen, indem Objekte auf verschiedene Arten behandelt werden können, ohne ihren konkreten Typ zu kennen. Java nutzt Polymorphismus intensiv in der Standardbibliothek und in Design Patterns.